

Trasformata di Burrows-Wheeler col Client-Server

Autore: Alberto Pelino

Istituto di Istruzione Superiore “Archimede” - Treviglio

Abstract:

Dopo aver capito il lavoro che mi è stato assegnato ho iniziato a ragionare sulla logica del codice in Python, dopo qualche ora di lavoro si è riuscito a ottenere il risultato pensato.

La parte iniziale del codice si basa sul calcolo della trasformata dopo di che si scrive la parte relativa a aprire la connessione al socket, dopo aver scritto tutto il codice per preparare il server alla connessione, inizio la parte relativa al client dove servirà per prendere i dati in input e restituirli.

Introduzione

In questa relazione spiego come ho fatto a trovare la Trasformata di Burrows-Wheeler con i socket , partendo dal linguaggio che ho usato fino alla spiegazione del codice.

Come linguaggio ho usato il linguaggio Python molto comodo e intuitivo perché è un linguaggio a alto livello, cioè un linguaggio vicino al linguaggio umano, come software per scrivere il codice ho usato l'IDE Visual Studio Code.

Materiale e metodi

Per programmare ho usato Visual Studio Code, utilizzando due classi, importando la libreria “socket”. I metodi usati sono:

- Ordinamento Bubble Sort

= Per ordinare la lista di parole utilizzo l'ordinamento Bubble Sort, che confronta due parametri e li riordina in modo che quello più piccolo vada primo.

```
for i in range(dimensione_lista-1):
    for j in range(0, dimensione_lista - i - 1):
        if lista[j] > lista[j + 1]:
            lista[j], lista[j + 1] = lista[j + 1], lista[j]
```

- `.socket()`

= Questo metodo si utilizza sia sulla parte client e sia sulla parte server, serve a creare un nuovo socket, che prepara la struttura di comunicazione.

Vuole come parametro la famiglia di indirizzi e il tipo di socket.

```
socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

- `.bind()`

= Associa l'oggetto socket appena creato all'indirizzo address, si utilizza solo nel server. Vuole come parametro l'indirizzo address.

```
s.bind((HOST, PORT))
```

- `.listen()`

= Abilita il socket a ricevere le connessioni, si utilizza solo nel server. Vuole come parametro il numero massimo di connessioni, di default ha 1.

```
s.listen()
```

- `.accept()`

= Mette il socket in attesa di nuove connessioni e restituisce una tupla, si utilizza solo nel server.

```
conn, addr = s.accept()
```

- `.recv()`

= Riceve dati dal socket restituendo in codice binario, servirà decodificarlo, si utilizza sia nel server che nel client.

```
data = conn.recv(1024)
```

- `.sendall()`

= Invia dati all'indirizzo dove il socket è connesso, si utilizza sia nel server che nel client. Vuole come parametro dati in byte.

```
conn.sendall(risultato.encode())
```

- `.connect()`

= Tenta di connettersi all'indirizzo specificato, si utilizza sia nel server che nel client. Vuole come parametro l'address.

```
s.connect((HOST, PORT))
```

- `.encode()`

= Si usa per la codifica da stringa a stringa binaria, si utilizza sia nel client e nel server.

```
.encode()
```

- `.decode()`

= Si usa per la codifica da stringa binaria a stringa, si utilizza sia nel client e nel server.

```
.decode ()
```

Risultati e discussione

Il codice ha fatto quello che ci aspettavamo cioè prendere che il client prende in input una frase che viene mandata al server, a questo punto il server esegue il codice per calcolare la trasformata applicando l'ordinamento lessicografico delle rotazioni della stringa, infine viene mandata al client la parola trasformata che la restituisce all'utente.

Conclusione

L'obiettivo di questo progetto è stato raggiunto grazie alle implementazioni di vari funzioni diverse che hanno permesso di realizzare correttamente i passaggi partendo dal collegamento tra client e server fino al codice per calcolare la trasformata.

Bibliografia (?)

Per ricavare le informazioni per il codice ho consultato [wikipedia](https://en.wikipedia.org/) per capire il funzionamento della trasformata.